

## POCKETSMART AI: PROJECT PLANNING & EXECUTION SCHEDULE

### 1. Executive Summary & Project Objectives

The Project Planning Phase establishes the structured roadmap, milestone schedules, resource allocations, and risk mitigation strategies for **PocketSmart AI**. The planning framework was formulated to guide development through iterative development cycles, ensuring modular decoupling of core FastAPI services, Google Gemini generative AI integrations, and responsive client-side Jinja2 interfaces within predefined academic deadlines.

### 2. Project Methodology: Agile Scrum Framework

To maintain high responsiveness to algorithmic adjustments, API updates, and UI changes, the project adopted the **Agile Scrum Methodology**:

- **Sprint Cycles:** Two-week iterative sprint cycles with defined deliverables.
- **Continuous Integration:** Immediate testing of newly integrated routes (/home-budget, /party-budget, /jewelry-budget) with FastAPI's interactive OpenAPI documentation (/docs).
- **Milestone Reviews:** Systematic checkpoint validation covering ideation, architectural design, prototyping, testing, and documentation packaging.

### 3. Work Breakdown Structure (WBS)

1.0 POCKETSMART AI INITIATIVE | |— 1.1 Requirements & Ideation Phase | |— 1.1.1 Problem statement definition & stakeholder empathy mapping | |— 1.1.2 Functional & Non-Functional requirement cataloging | |— 1.1.3 Technology stack

selection (FastAPI, Python 3.12, Gemini AI) | └─ 1.2 System Design & Prototyping | └─ 1.2.1 High-level architectural design and DFD formulation | └─ 1.2.2 Wireframing modular budget planners (Home, Party, Jewelry) | └─ 1.2.3 Data persistence schema design (users.json & session store) | └─ 1.3 Core Engineering & API Integration | └─ 1.3.1 Asynchronous FastAPI backend setup and route modularization | └─ 1.3.2 Google Gemini AI integration and prompt engineering | └─ 1.3.3 Deterministic heuristic fallback engine implementation | └─ 1.3.4 Dynamic affiliate e-commerce search query generators | └─ 1.3.5 OAuth2 JWT authentication & Bcrypt security integration | └─ 1.4 Frontend Construction & Template Rendering | └─ 1.4.1 Jinja2 template engineering with Bootstrap 5 | └─ 1.4.2 Client-side asynchronous fetch event handling | └─ 1.4.3 Dynamic UI result card & shopping badge styling | └─ 1.5 Quality Assurance & Performance Testing | └─ 1.5.1 API endpoint stress testing & latency evaluation | └─ 1.5.2 Fallback resilience verification under simulated network loss | └─ 1.5.3 Cross-browser responsiveness and UI validation | └─ 1.6 Documentation & Final Submission Packaging | └─ 1.6.1 Academic milestone PDF generation | └─ 1.6.2 Video demonstration recording & code repository archiving | └─ 1.6.3 Final review against submission criteria

4. Project Schedule & Sprint Timeline (Milestone Roadmap)

| Sprint / Phase | Focus Area                      | Start Date | End Date | Primary Deliverables    | Status    |
|----------------|---------------------------------|------------|----------|-------------------------|-----------|
| Sprint 1       | Requirement Analysis & Ideation | Week 1     | Week 2   | Problem statements, DFD | Completed |

|                 |                                   |        |         |                                                                              |           |
|-----------------|-----------------------------------|--------|---------|------------------------------------------------------------------------------|-----------|
|                 |                                   |        |         | diagrams, and tech stack finalization.                                       |           |
| <b>Sprint 2</b> | Architecture & UI Prototyping     | Week 3 | Week 4  | Bootstrap 5<br>Jinja2 layout templates and system architecture schemas.      | Completed |
| <b>Sprint 3</b> | Core Backend & Router Engineering | Week 5 | Week 6  | FastAPI route setup, JWT authentication, and JSON persistence engine.        | Completed |
| <b>Sprint 4</b> | AI Integration & Fallback Shield  | Week 7 | Week 8  | Gemini AI API calls, fallback algorithms, and e-commerce deep link builders. | Completed |
| <b>Sprint 5</b> | Testing, Auditing & Optimization  | Week 9 | Week 10 | Latency benchmarking, cross-platform checks, and bug fixes.                  | Completed |

|                 |                               |         |         |                                                                         |           |
|-----------------|-------------------------------|---------|---------|-------------------------------------------------------------------------|-----------|
| <b>Sprint 6</b> | Documentation & Demonstration | Week 11 | Week 12 | Video demonstration recording, screenshot catalogs, and milestone PDFs. | Completed |
|-----------------|-------------------------------|---------|---------|-------------------------------------------------------------------------|-----------|

5. Resource Allocation & Responsibility Matrix (RACI)

To guarantee balanced execution, tasks were distributed across key operational roles:

- **Backend & System Architecture:** Managing ASGI configurations, FastAPI routing controllers (routes/planners.py), and JSON database persistence.
- **AI & Integration Engineering:** Configuring Google Generative AI SDK, structuring system prompts, and programming the heuristic fallback engine.
- **Frontend Design & UX:** Crafting Jinja2 templates, Bootstrap 5 responsive cards, forms, and client-side JavaScript fetch handlers.
- **Security & Authentication:** Implementing OAuth2 standards, JWT bearer tokens, and Bcrypt password hashing algorithms.
- **Quality Assurance & Documentation:** Coordinating functional testing, collecting UI execution logs, and assembling academic documentation.

6. Risk Management & Mitigation Matrix

|                        |                      |                   |                                      |
|------------------------|----------------------|-------------------|--------------------------------------|
| <b>Identified Risk</b> | <b>Risk Severity</b> | <b>Likelihood</b> | <b>Proactive Mitigation Strategy</b> |
|------------------------|----------------------|-------------------|--------------------------------------|

|                                            |        |        |                                                                                                                 |
|--------------------------------------------|--------|--------|-----------------------------------------------------------------------------------------------------------------|
| <b>External LLM Latency / Quota Limits</b> | High   | High   | Implemented an instantaneous algorithmic fallback calculation engine ensuring 100% uptime and 200 OK responses. |
| <b>Library Version Incompatibilities</b>   | Medium | Medium | Standardized on Python 3.12 with fallback password verification handling Passlib/Bcrypt version variances.      |
| <b>Session Loss on Server Reloads</b>      | Medium | High   | Replaced transient in-memory dictionaries with persistent file storage via <code>users.json</code> .            |
| <b>Cross-Device UI Distortions</b>         | Low    | Low    | Utilized Bootstrap 5 fluid container grids and viewport-relative scaling units.                                 |